

CHAPTER 2

LITERATURE REVIEW

2.1 How to read this chapter

This chapter makes one argument: that a safety requirement on a learned manipulation policy is better expressed as a constraint with a stated budget than as a penalty term inside the reward, and that the published work closest to this thesis leaves two questions about that claim unanswered. Everything here supports, qualifies or positions that argument.

It proceeds in four movements. Sections 2.2 to 2.5 build the machinery: why reward shaping is an awkward vehicle for safety, what a constrained Markov decision process is, and which deep constrained algorithm this thesis uses. Sections 2.6 to 2.8 turn to robotics and set the reviewed studies side by side so their methods can be compared directly. Sections 2.9 and 2.10 treat the two quantities on which the work turns: the manipulability measure that defines its safety constraint, and the seed-to-seed variance that governs how its results must be reported. Sections 2.11 and 2.12 state the gap and close. A reader in a hurry can go straight to Section 2.11, which is what Chapter 1 refers back to and what Chapter 4 answers.

Table 2.1. Section map for this chapter.

§	Subject	Principal sources
2.2	Why safety resists reward shaping	[?], [?]
2.3	Safe RL as a field, and its taxonomy	[?], [?]
2.4	The constrained Markov decision process	[?]
2.5	Deep constrained policy optimisation	[?], [?], [?], [?]
2.6	Safety in robot learning	[?], [?]
2.7	Deep RL on manipulators and on the UR5	[?], [?], [?]
2.8	Experimental methods of the reviewed studies	all nine experimental papers
2.9	Manipulability and singularity avoidance	[?], [?]
2.10	Reproducibility and seed variance	[?]
2.11	Research gap and positioning	[?]

The findings the chapter reaches are collected below, so that the sections that follow read as evidence for them instead of as a survey in search of a conclusion. Each is picked up again in Section 2.12.

Key findings of this chapter

1. A penalty weight and a cost budget are not two spellings of the same instruction. The weight fixes an exchange rate between safety and task progress that the designer must guess in advance; the budget states a limit in the units of the hazard and can be audited after training [?].
2. The constrained Markov decision process is the prevailing formalism for this [?], and its Lagrangian relaxation turns the guessed weight into a dual variable the optimiser sets from the observed violation [?].
3. PPO-Lagrangian is preferred to CPO on published evidence. On the eighteen-environment Safety Gym slate, CPO left a normalised violation of 0.593 against 0.026 for PPO-Lagrangian [?].
4. The Lagrange multiplier under plain dual ascent behaves as an integral controller and oscillates against the cost with a quarter-cycle lag [?]. Any multiplier reported in Chapter 4 is a controller mid-flight, not a converged constant.
5. Yoshikawa’s measure $w = \sqrt{\det(JJ^T)}$ is the volume of the manipulability ellipsoid and the product of the Jacobian’s singular values [?]. Prior learning work puts it in the reward with a hand-set weight, in one case $\lambda_3 = 0.05$ [?]. This thesis puts it in a constraint.
6. Random seed alone can produce statistically distinct outcome distributions for one algorithm on one task, at $t = -9.0916$, $p = 0.0016$ [?]. Means over three seeds, which is common practice here, are weak evidence for a safety claim.
7. The closest published comparison of PPO against its constrained variant on a manipulator [?] reports no control arm isolating the cost critic from the constraint, no seed-to-seed dispersion, and no result at more than one budget. These are the three gaps this thesis fills.

2.2 The safety requirement in learned manipulation

A robot manipulator that learns its own control policy poses a problem that one running a hand-written controller does not. A classical controller can be inspected, bounded and argued about before the machine is switched on. A policy obtained by reinforcement learning is the outcome of an optimisation run, and is known only through the statistics of the trajectories it produces. When such a policy is to be deployed on a physical arm sharing its workspace with people, equipment or the workpiece, what it does in the configurations it visits only rarely is

not a secondary concern. It is the condition on which deployment depends at all.

The difficulty is that reinforcement learning offers a single channel through which a designer expresses what the agent should and should not do: the reward function. Safety requirements entered through that channel become penalty terms, and a penalty term needs a weight. That weight is a trade-off rate between task performance and safety, fixed before the designer knows what either will be worth, and it has no natural units. Achiam *et al.* [?] make this argument directly in motivating constrained policy optimisation: for systems interacting physically with or around humans, it is more natural to specify a reward function *and* a set of constraints than to encode both into one scalar objective. Ray *et al.* [?] put it operationally. Too small a penalty and the agent learns unsafe behaviour; too severe a penalty and it may fail to learn at all. A fixed trade-off also cannot respond to the fact that the pressure the reward exerts on the cost changes as the policy improves.

Keeping the requirement outside the reward and imposing it as a constraint with a budget avoids this. A budget can be stated in advance, in the units of the quantity being constrained, and audited afterwards against what the trained policy actually spent. That distinction, that *a shaped reward requires the designer to guess a weight while a constraint requires only that they state a limit*, is the premise of this thesis. Section 2.9 shows it separating two otherwise similar bodies of work on singularity avoidance, where the guessed weight in the nearest such study is a bare 0.05 with no stated derivation.

2.3 Safe reinforcement learning

Safe reinforcement learning studies methods that optimise a policy while respecting some notion of acceptable behaviour during learning, at deployment, or both. The organising survey is that of García and Fernández [?], which splits the literature by where the safety requirement is inserted. One branch modifies the *optimisation criterion*, changing the objective the agent maximises through a risk-sensitive transformation, a worst-case criterion, or explicit constraints. The other modifies the *exploration process*, leaving the objective intact but restricting the actions the agent may try, typically using external knowledge such as a teacher, a demonstration set or a backup controller.

Table 2.2. The two branches of safe reinforcement learning, after [?].

Branch	Mechanism	Evaluated on
Modify the optimisation criterion	Risk-sensitive or worst-case objectives; explicit constraints on expected cumulative cost	The trained policy, on held-out episodes
Modify the exploration process	Teacher guidance, demonstrations, backup controllers, action shielding	Violations accumulated during training

This thesis belongs to the first branch, and to its constrained-criterion sub-family. The distinction matters for reading Chapter 4: second-branch methods aim to make the learning process itself safe and are evaluated on violations accumulated *during* training, whereas first-branch methods aim to produce a policy that satisfies a requirement once trained, and are properly evaluated on the frozen policy. This work follows the latter convention, and Section 4.1 explains why an earlier version of these results, which read safety figures from training-time telemetry, was withdrawn. The more recent review by Gu *et al.* [?] organises the post-deep-learning field around five open problems, of which the benchmark question is why Chapter 4 reports episodic cost against a declared budget rather than an ad hoc safety score. The same review reports that the CMDP has become the prevailing formalism for safe reinforcement learning, which places this work inside the mainstream of the field and is why the CMDP is adopted here without further argument.

2.4 The constrained Markov decision process

The formal object underlying that branch is the constrained Markov decision process, developed in its modern form by Altman [?]. A standard Markov decision process is the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ over a state space, an action space, a transition kernel, a reward function and a discount factor. A CMDP augments this with a cost function that scores each transition for hazard exactly as the reward scores it for progress, and a budget, and it asks for the policy maximising expected return subject to expected accumulated cost staying within that budget. Chapter 3 states the problem formally at Equation (3.1), in the notation of the implementation, and it is not repeated here. That division holds throughout: this chapter carries the equations of the work being reviewed, and Chapter 3 carries those of the method being built.

Two properties matter for the argument. The safety requirement is expressed in the units of the cost rather than of reward, so it can be set from a measurement of the system instead of a tuning exercise, which is what the calibration of Section 3.9 depends on. And the constrained problem admits a Lagrangian relaxation, folding the constraint into the objective through a multiplier that is itself optimised:

$$\max_{\theta} \min_{\lambda \geq 0} \mathcal{L}(\theta, \lambda) = J_r(\pi_{\theta}) - \lambda(J_c(\pi_{\theta}) - d). \quad (2.1)$$

The duality theory licensing that treatment is the contribution of [?], and it is what makes the algorithms of the next section possible: instead of solving a constrained problem directly, they solve a sequence of unconstrained ones in which the penalty weight is driven by the observed violation. That point is easily lost, because a Lagrangian method does end up optimising a weighted sum of reward and cost, which superficially resembles reward shaping. The difference is that the weight is not a hyperparameter. It is a dual variable that rises while

the budget is exceeded and falls once it is met. A useful test is to ask what the designer would change if the hazard became twice as costly to the operator. Under reward shaping they would retune a weight and retrain until the behaviour looked right; under a constraint they would halve the budget and let the optimiser find the weight.

2.5 Constrained policy optimisation in deep reinforcement learning

The baseline algorithm throughout this thesis is proximal policy optimisation [?], which limits the size of each policy update by clipping the probability ratio between the candidate policy and the one that collected the data, so that a large policy change can never be rewarded. Chapter 3 gives the clipped surrogate objective at Equation (3.3) and explains why the clip does the work of a trust region without a constrained optimisation. What matters here is the consequence: PPO attains much of the reliability of trust-region methods at a fraction of their implementation complexity, which is why it is also the algorithm most commonly extended when a constraint is added. Both grasping studies of Section 2.7 use it, at $\epsilon = 0.2$ in the UR5 case [?], and so does the constrained work nearest this thesis [?].

Constrained policy optimisation (CPO) [?] first brought the CMDP into deep reinforcement learning at scale, replacing the unconstrained trust-region update with

$$\pi_{k+1} = \arg \max_{\pi \in \Pi_\theta} J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) \leq d, \quad D(\pi, \pi_k) \leq \delta, \quad (2.2)$$

solved approximately at every step so that each update both improves the return and maintains near-constraint satisfaction. Its significance here is as much conceptual as algorithmic: it established the constrained formulation as a practical alternative to reward shaping for high-dimensional continuous control, and it supplies the argument quoted in Section 2.2.

The algorithm actually implemented is the Lagrangian variant of PPO, as specified in the Safety Gym benchmark report of Ray *et al.* [?]. Its constrained agents are trained by gradient ascent on θ and descent on λ in Equation (2.1), at a cost limit of $d = 25$ throughout, over 1000-step episodes and three random seeds per environment. That budget of 25 is inherited by Chapter 4 rather than chosen freshly, which removes one degree of freedom from the present design. The second reason to cite that report is empirical, and is why PPO-Lagrangian was preferred to CPO.

CPO retains far more return than either Lagrangian method, but leaves a normalised violation of 0.593, removing only about forty percent of the unconstrained agent’s violations where the Lagrangian methods remove roughly ninety-seven percent. Ray *et al.* attribute the shortfall to approximation error in CPO’s analytic step, and note that the result contradicts the one reported in [?] on easier environments. Adopting CPO would therefore have required a justification this work does not have.

Table 2.3. Normalised end-of-training metrics on the eighteen-environment Safety Gym slate, three seeds per environment, from [?]. Normalised against unconstrained PPO, so 1.0 is the baseline and lower is safer.

Algorithm	Return $\bar{J}_r$	Violation $\bar{M}_c$	Cost rate $\bar{\rho}_c$
PPO (unconstrained)	1.000	1.000	1.000
TRPO (unconstrained)	1.094	1.132	1.004
PPO-Lagrangian	0.240	0.026	0.245
TRPO-Lagrangian	0.331	0.018	0.265
CPO	0.784	0.593	0.646

Lagrangian methods carry a characteristic weakness, and this thesis meets it directly in Chapter 4. Stooke *et al.* [?] identify the cause precisely: plain dual ascent on λ accumulates the running constraint violation, which is integral control on the constraint and nothing else. An integral controller with no proportional or derivative action overshoots, so the cost and the multiplier oscillate against one another with a quarter-cycle lag, as Figure 2.1 shows.

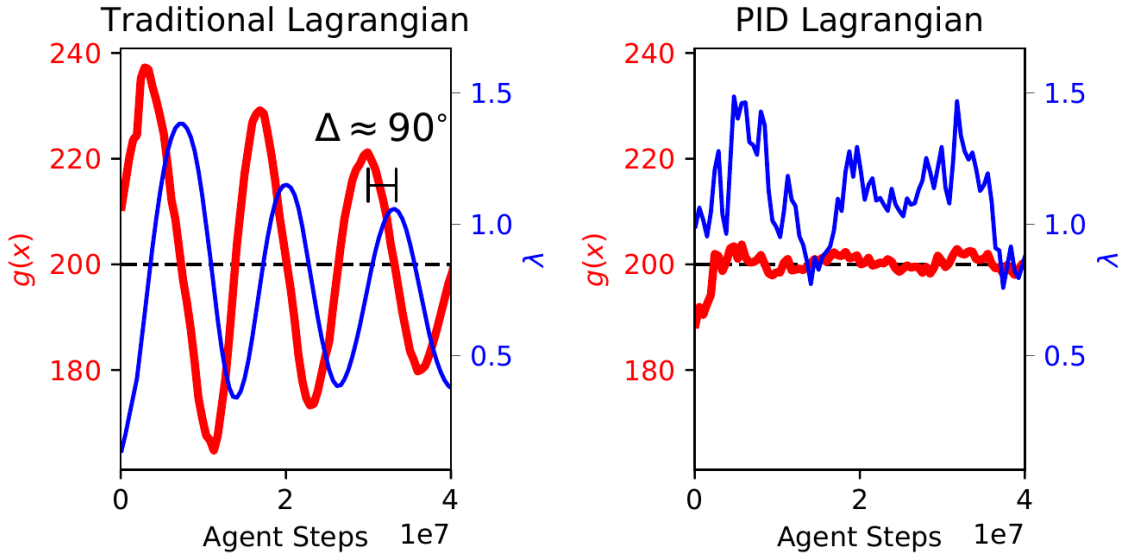


Figure 2.1. Cost $g(x)$ and multiplier λ under the traditional Lagrangian update (left) and under PID control (right). From [?], Fig. 1.

Their remedy treats the multiplier as a control input under full PID control,

$$\Delta_k = J_c(\pi_k) - d, \quad I_k = (I_{k-1} + \Delta_k)_+, \quad \partial_k = (J_c(\pi_k) - J_c(\pi_{k-1}))_+, \quad (2.3)$$

$$\lambda_k = (K_P \Delta_k + K_I I_k + K_D \partial_k)_+, \quad (2.4)$$

where $(\cdot)_+$ is projection onto the non-negative reals and $K_P = K_D = 0$ recovers the traditional method exactly. Two implementation details are inherited by this thesis. They maintain a *separate* cost-value approximator alongside the reward-value approximator instead of folding

$r + \lambda c$ into a single critic, on the grounds that λ may change quickly, and they rescale the policy gradient by $1/(1 + \lambda)$ so that a large multiplier does not enlarge the effective step size. The first of those is why the constrained agent here carries a network its baseline does not, which is in turn why a control arm is needed at all; Section 2.11 returns to it. Their analysis is used in Section 4.6 to read the multiplier values observed at the end of training, and the PID form is identified in Chapter 6 as the natural extension of this work. It is worth noting that [?] names its PPO-based agent CPPO, the same abbreviation this thesis uses. The benchmark suites have since been superseded: Safety Gymnasium [?] replaces the original environments and ships a library of safe algorithms under a common protocol, and its reporting conventions, episodic cost against a stated budget and violation rates measured per episode, are those adopted in Chapter 4.

2.6 Safety in robot learning

The literature reviewed so far approaches safety from the machine-learning side. Robotics approaches it from the control side, with a vocabulary of invariant sets, barrier certificates and stability guarantees that developed independently. Brunke *et al.* [?] reconcile the two into a common framework spanning learning-based control through to safe reinforcement learning. That reconciliation is needed here because this thesis sits across the boundary: the quantities it constrains (the manipulability measure, the distance to a joint limit, the clearance above the work surface) are control-theoretic constructs describing the kinematic state of the arm, while the mechanism enforcing them is a cost critic and a dual variable.

The survey also draws a distinction that limits what may be claimed here. Methods enforcing a constraint in expectation, as a Lagrangian method does, bound *expected* exposure to unsafe configurations; they do not bound the severity of any individual excursion. Methods supplying a safety certificate, such as a control barrier function filtering the commanded action, offer the latter guarantee but require a model. Section 4.5 reports a counter-result of exactly this shape, and Chapter 6 argues the two mechanisms address different halves of the problem. Within manufacturing specifically, Elguea-Aguinaco *et al.* [?] review reinforcement learning for contact-rich manipulation from 2017 onward, and their survey is used in Chapter 3 to establish which design choices made here follow established practice.

2.7 Deep reinforcement learning for robotic manipulation

Applications of deep reinforcement learning to manipulation divide broadly by algorithm family. Shahid *et al.* [?] compare PPO against soft actor-critic on a grasping task with a Franka Emika Panda in MuJoCo, splitting the task into a reaching and a lifting phase with a separate dense reward for each and adding penalties on joint-position proximity, joint acceleration and obstacle approach by fine-tuning rather than all at once. The policy runs at

250 Hz against a 500 Hz joint controller and trains for 10^7 steps. Evaluated on the physical Panda with no real-world training data, it succeeds ten times out of ten on each of four objects. That study is the methodological template for an on-policy against off-policy comparison, it demonstrates zero-shot transfer to hardware, and it is the precedent for the off-policy arm planned for this study and ultimately not trained (Section 4.9).

Work specific to the UR5 is less common. Ferreira and Barbosa [?] train a UR5 with a Robotiq 2F-85 gripper to grasp randomly positioned objects in PyBullet at 240 Hz, comparing soft actor-critic against PPO under identical conditions with domain randomisation. Their scene, in Figure 2.2, is the closest published analogue to the environment of this thesis in both robot and gripper.

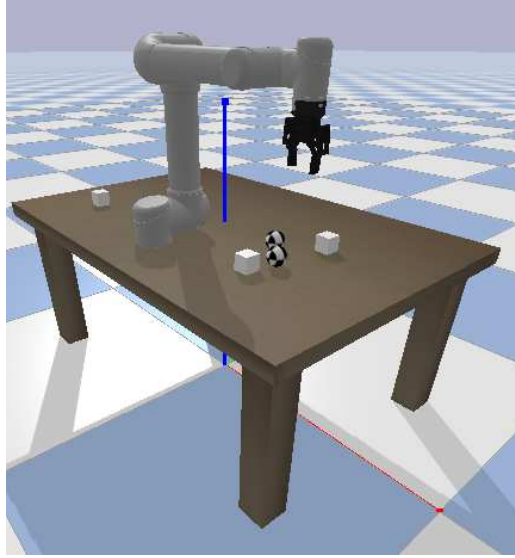


Figure 2.2. A UR5 with a Robotiq 2F-85 gripper in PyBullet. From [?], Fig. 1, CC BY 4.0.

Their reward is shaped,

$$R(s, a) = -\lambda \|p_g - p_o\|_2 + r_s \mathbb{I}_{\text{grasp}} + r_p \mathbb{I}_{\text{pose}}, \quad (2.5)$$

with $\lambda = 0.1$ weighting the gripper-to-object distance penalty, $r_s = 10$ for a grasp held at least 0.5 s and $r_p = 5$ for correct placement orientation. Their own ablation shows how much rests on those hand-set numbers: removing the distance term costs ten percentage points of success, the grasp reward twelve, the pose reward eight, and reducing the function to its two sparse indicators eight. Both agents trained for 100,000 timesteps at a learning rate of 3×10^{-4} , batch size 64, two-layer 256-unit MLP. They report grasp success of 87 % for SAC and 82 % for PPO, with post-grasp placement success of 75 % and 68 %. Those are the closest published figures for a learned UR5 grasping policy, with one qualification that must be carried into any comparison: their task requires a full physical grasp in which the fingers close and the contact must be stable, whereas this thesis abstracts the grasp as a weld

for the reasons given in Section 3.5. The success rates of Chapter 4 are therefore not directly comparable to theirs and are not presented as though they were.

On the safety side, Xia *et al.* [?] apply deep reinforcement learning to proactive obstacle avoidance for a UR5e in a human-robot collaborative assembly cell, using an RGB-D camera and a wrist force-torque sensor with a soft actor-critic agent under a composite reward, and report a 93 % success rate in avoiding dynamic obstacles while still reaching the goal pose. Their work establishes that safe operation of this specific manipulator is an active industrial concern and not an academic construction, which is the premise on which the practical relevance of this thesis rests. It also illustrates the alternative design route: where this thesis expresses safety as a budget on a kinematic cost, they express it as another term inside a composite reward, and 93 % is a success rate rather than a bound on anything.

2.8 Experimental methods of the reviewed studies

Describing each study in turn makes it hard to see what the field holds fixed and what it varies. Table 2.4 therefore sets the experimental apparatus side by side. It covers only the papers reporting their own experiments; the three surveys are excluded.

Three patterns in that table shape decisions made in Chapter 3. The cost limit $d = 25$ recurs: Ray *et al.* set it as the Safety Gym reference configuration [?], Khan *et al.* inherit it unchanged along with $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.97$, a penalty learning rate of 5×10^{-2} and a penalty initialised at 1.0 [?], and this thesis inherits it in turn. A number reused across three studies is not thereby correct, but it does mean the budget in Chapter 4 is a convention and not a free parameter tuned to flatter the result. Network capacity is also uniformly small, two hidden layers of 64 to 256 units across all of them, and none of the reported outcomes appears limited by it, which is worth knowing before attributing any difference in this thesis to architecture.

The third pattern is one of absence, and it matters most. Of the eight experimental studies, three report a seed count and five do not. None of the three reports per-seed dispersion; they report a mean, or a shaded band whose composition is not decomposed. Section 2.10 takes up what follows.

2.9 Manipulability and singularity avoidance

The constraint this thesis is built around is a floor on manipulability. One qualification belongs here rather than later, because it changes how this section should be read. The cost function of Chapter 3 scores three hazards rather than one, and joint-limit proximity is genuinely active alongside manipulability, with the collision term inactive throughout. Across the five unconstrained baseline runs the manipulability term carries about 78 % of the realised

Table 2.4. Experimental apparatus of the reviewed studies. A dash means the paper does not state a seed count.

Study	Platform	Simulator	Algorithms	Seeds	Headline result
Ray <i>et al.</i> [?]	Point, Car, Doggo	MuJoCo / Safety Gym	PPO, TRPO, both Lagrangian, CPO	3	PPO-Lag. violation 0.026 vs CPO 0.593 (normalised)
Stooke <i>et al.</i> [?]	Doggo	Safety Gym	CPPO with PID multiplier	–	Cost oscillation damped; violations reduced
Henderson <i>et al.</i> [?]	MuJoCo locomotion	OpenAI Gym	TRPO, PPO, DDPG, ACKTR	5 (and 10)	Seed alone gives distinct distributions, $p = 0.0016$
Shahid <i>et al.</i> [?]	Franka Panda	MuJoCo	PPO, SAC	–	100 % grasp success, zero-shot on hardware
Ferreira and Barbosa [?]	UR5 + 2F-85	PyBullet	PPO, SAC	–	SAC 87 % vs PPO 82 % grasp success
Xia <i>et al.</i> [?]	UR5e	Custom + ROS	SAC	–	93 % dynamic obstacle avoidance
Shen <i>et al.</i> [?]	4-DOF planar	Custom	SAC in Jacobian null space	5	96.8 % vs 77.4 % avoidance; higher mean w
Khan <i>et al.</i> [?]	Franka Panda	Safety Gym	PPO, cPPO	–	cPPO cost 12.32 vs PPO 17.76 (SOPM)
This thesis	UR5e	Isaac Sim	PPO (baseline), cPPO $d = 25$, cPPO $d = 15$	5	Chapter 4

episodic cost on average and joint-limit proximity about 22 %, but that average is unstable: three of the five runs spend their whole budget on near-singular configurations and never approach a joint limit, while one spends most of its budget on joint-limit proximity instead (Section 3.9). Manipulability is therefore the framing constraint, the one that motivates the design, and on the mean the dominant one, but it is not the only term the budget is spent on and it does not dominate on every seed. This section develops the measure carefully because the argument of the thesis rests on it and because the difference between this work and its nearest neighbour is entirely a difference in where that quantity is placed.

2.9.1 The measure and what it means

Consider a manipulator with n joints, joint vector $\theta \in \mathbb{R}^n$, whose task is described by $m \leq n$ manipulation variables $\mathbf{r} \in \mathbb{R}^m$. Differentiating the forward kinematics gives

$$\dot{\mathbf{r}} = J(\theta) \dot{\theta}, \quad J(\theta) \in \mathbb{R}^{m \times n}, \quad (2.6)$$

with J the manipulator Jacobian. Yoshikawa [?] defines a configuration θ^* to be *singular* when $\text{rank } J(\theta^*) < m$. At such a configuration the end-effector cannot be moved instantaneously along at least one Cartesian direction, whatever joint velocities are commanded. As a scalar index of distance from that condition he proposes

$$w(\theta) = \sqrt{\det(J(\theta)J(\theta)^\top)}. \quad (2.7)$$

Three properties established in [?] explain why this particular combination is the right one, and each is used somewhere in this thesis. First, writing the singular value decomposition $J = U\Sigma V^\top$ with singular values $\sigma_1 \geq \dots \geq \sigma_m \geq 0$, the measure is their product,

$$w = \sigma_1 \sigma_2 \dots \sigma_m, \quad (2.8)$$

so it collapses as soon as any single σ_i does. It is sensitive to the loss of *one* direction of motion even when the arm remains perfectly capable in every other direction, which is exactly the failure that matters. Second, the set of end-effector velocities reachable from unit-norm joint velocities is an ellipsoid with principal axes $\sigma_1 u_1, \dots, \sigma_m u_m$, and w is proportional to its volume, so the index measures how much end-effector motion a unit of joint effort buys. Third, when $m = n$ Equation (2.7) reduces to $w = |\det J|$; the UR5e has six joints controlling a six-dimensional pose, so this is the form that applies here.

Yoshikawa's simplest worked example makes the quantity concrete. For a planar two-link arm with link lengths ℓ_1, ℓ_2 and joint angles θ_1, θ_2 ,

$$w = \ell_1 \ell_2 |\sin \theta_2|, \quad (2.9)$$

maximised at $\theta_2 = \pm 90^\circ$ and vanishing when the arm is fully extended or fully folded, as Figure 2.3 shows. He notes that a human arm, treated as a two-link mechanism, satisfies $\ell_1 \approx \ell_2$ and is habitually used with the elbow near a right angle, so people appear to adopt high-manipulability postures without being told to. The intuition to keep is that a straightened arm is a weak arm: it can still push along its own length, but it can barely move sideways, and a controller asking it to do so will demand very large joint rates.

That consequence is why a floor on w counts as a safety constraint rather than a performance target. Near a singularity the inverse mapping is ill-conditioned, so a small commanded Cartesian velocity maps to a large joint velocity. On a real UR5e whose joints are capped at 180°s^{-1} , that appears either as a violent motion or as a protective stop. Yoshikawa anticipates this by normalising the Jacobian by the per-joint maximum velocities when those differ, which for the UR5e they do not, since all six joints share the same ceiling. The unnormalised

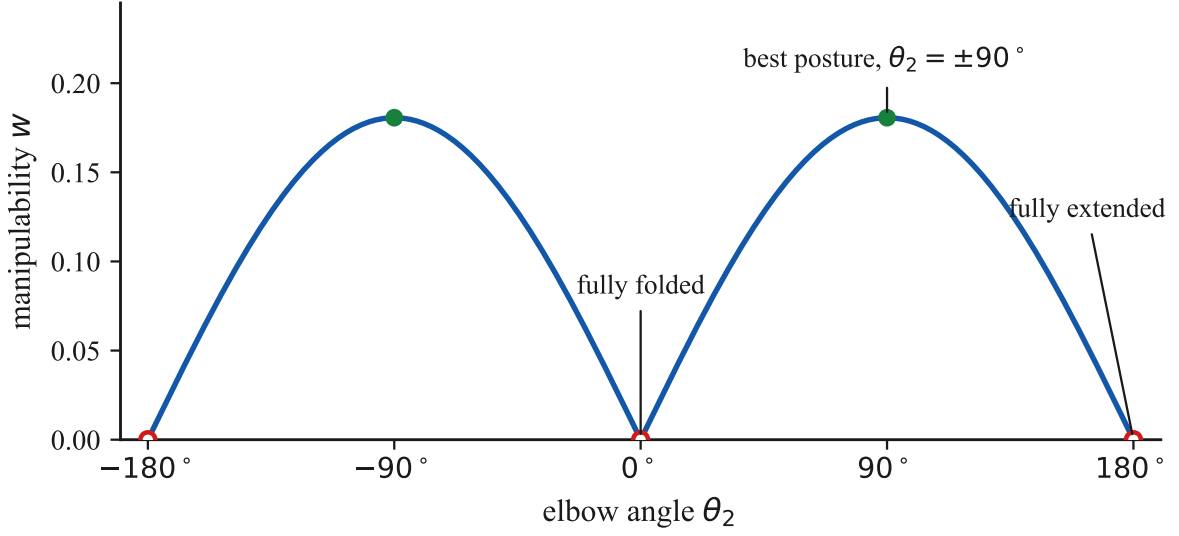


Figure 2.3. Manipulability of a planar two-link arm with equal links, from Equation (2.9). It falls to zero at both the folded and the extended posture and peaks at a right angle.

form is therefore used directly in Chapter 3, where the cost term is an excursion below a floor $w_{\min}$, and where Section 3.9 records the recalibration of that floor from 0.045 to 0.06 after the baseline policy’s natural operating distribution was measured.

2.9.2 The same measure, placed differently

Shen *et al.* [?] address reactive obstacle avoidance for redundant manipulators by learning self-motion in the null space of the Jacobian, so the commanded task velocity is preserved exactly while the learned action redistributes the remaining freedom:

$$\dot{q}_N = (I - J^\dagger J)\phi, \quad \dot{q}_D = J^\dagger \dot{x}_D, \quad (2.10)$$

with ϕ the learned action and $J^\dagger$ the pseudo-inverse. Their agent is soft actor-critic and their reward is the weighted sum

$$r = \lambda_1 r_o + \lambda_2 r_a + \lambda_3 r_m, \quad r_o = \sum_{i=1}^n \min\left(\frac{\|d_i\|}{d_s} - 1, 0\right), \quad r_m = \sqrt{\det(JJ^\top)}, \quad (2.11)$$

in which r_o penalises obstacle proximity inside a safe distance d_s , r_a penalises null-space joint motion, and r_m is Yoshikawa’s measure entered directly as a reward term. Their reported weights are $\lambda_1 = 1$, $\lambda_2 = 0.2$ and $\lambda_3 = 0.05$, with a flat penalty of -10 for a collision or joint-limit breach. The method works: over 1000 randomly generated obstacle configurations it raises avoidance success from 77.4 % to 96.8 % against a gradient-projection baseline in the two-obstacle scenario while holding a higher mean manipulability throughout (3.72

against 3.63), and it computes the null-space motion faster because a forward pass replaces an optimisation. Figure 2.4 shows what that looks like in configuration space.

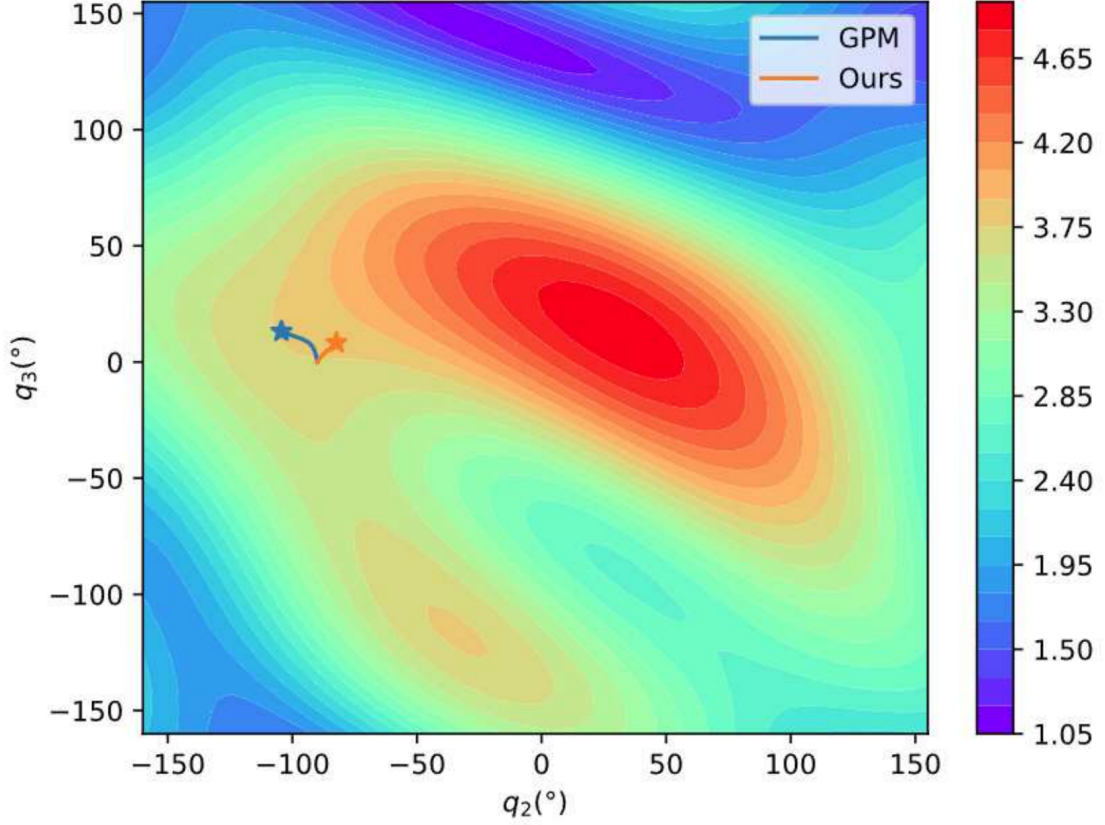


Figure 2.4. Manipulability over the q_2 - q_3 plane, with the paths taken by the baseline and by the learned policy from a common start. From [?], Fig. 12, CC BY 4.0.

The objective pursued there is the same as the one pursued here, and the quantity is identical down to the square root. The mechanism is not. Placing manipulability in the reward requires a weight relating a unit of w to a unit of task progress, and $\lambda_3 = 0.05$ is that weight. Nothing in the paper derives it, and nothing could, because there is no physical exchange rate between an ellipsoid volume and a metre of end-effector travel. It is a number that produced acceptable behaviour. Placing the same quantity in a constraint requires instead a budget, a statement of how much cumulative proximity to singularity an episode may incur, which can be measured from a baseline policy as Section 3.9 does and audited against the trained one as Section 4.5 does. Both approaches seek the same behaviour from the same measure. One obtains it by tuning a trade-off; the other by declaring a limit.

2.10 Reproducibility and seed variance

A final strand of the literature is methodological instead of algorithmic, and it bears on the central experimental finding of this work. Henderson *et al.* [?] examine reproducibility

in deep reinforcement learning and identify several sources of variation that are not the contribution being claimed: network architecture and activation function, reward scaling, choice of environment, the baseline codebase used, and the random seed. Their treatment of the last is unusually direct. They run ten trials of TRPO on HalfCheetah-v1 under one fixed hyperparameter configuration, varying nothing but the seed, split those ten arbitrarily into two groups of five, and average each group. If seed were noise around a stable mean the two curves would coincide. Figure 2.5 shows that they do not. A two-sample t -test across the training distribution gives $t = -9.0916$ and $p = 0.0016$: two groups drawn from the same algorithm, the same task and the same settings belong to statistically distinct distributions.

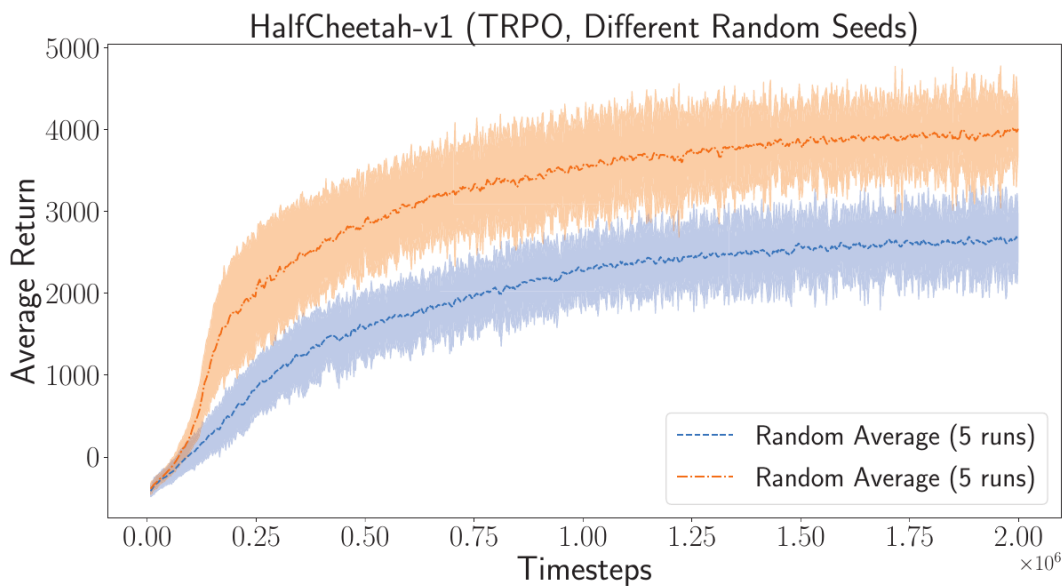


Figure 2.5. TRPO on HalfCheetah-v1, two groups of five random seeds under identical settings ($t = -9.0916$, $p = 0.0016$). From [?], Fig. 5.

Their bootstrap intervals give the size of the effect. On HalfCheetah-v1 the 95 % interval for DDPG runs from 3664 to 6574 about a mean of 5037, a span of more than half the mean, so two papers reporting single runs of the same algorithm on that environment could differ by a factor approaching two without either having done anything wrong. Their recommendation follows: report over multiple independent seeds with a measure of dispersion, treat comparisons with corresponding caution, and do not select the best of several runs. They decline to name a required number of trials, but note that averaging fewer than five is common in published work and is not defensible.

Set that against Table 2.4. Ray *et al.* use three seeds [?]; Shen *et al.* use five and plot the minimum-to-maximum band [?]; five of the eight experimental studies reviewed here, including the one nearest this thesis, do not state a seed count at all. That is not a criticism of any individual paper, since the convention is field-wide, but it is an observation about what the published safety numbers can support.

Three consequences follow, of different kinds. Procedurally, the protocol of Chapter 3 trains every arm on five independent seeds (1, 3, 4, 52 and 54) and reports dispersion across them rather than a single figure, justified by reference to [?] rather than by assertion, which is why this section sits in the literature review and not in the methodology. Interpretively, the seed-to-seed spread reported in Section 4.6 is an instance of a documented property of the method and is presented there as a measured quantity and not as a surprising discovery. And there is a consequence for what a safety claim is for. Henderson *et al.* are concerned with fair comparison between algorithms, which is a concern of authors. An integrator deploying a policy has a different one: they do not obtain the mean over training runs they will never perform, but the single policy they happened to train, on the single seed they happened to use. For that reader the dispersion is not an error bar around the result. It is the result. That reframing is what motivates the question Chapter 4 actually asks, which is whether a constraint tightens the spread of safety outcomes as well as shifting their centre.

2.11 Research gap and positioning

The nearest published analogue to this study is that of Khan *et al.* [?], which compares a model-free PPO baseline against its constrained variant for precision grasping on a Franka Panda in Safety Gym, and introduces a tactile feedback scheme to accelerate learning under safety compliance. Their agent is a two-layer 64-unit MLP actor-critic on an eight-dimensional observation, trained for 250 epochs of 2000 steps at the Safety Gym reference settings, including the cost limit of 25 this thesis also uses. On single-object precision manipulation they report a final average episodic cost of 12.32 for cPPO against 17.76 for PPO, and on the harder multi-object task 29.95 against 38.80, in both cases giving up a little reward for a substantially lower cost. It is the closest match in method, framing and intent, and this thesis continues that line of work rather than departing from it.

Three gaps are nonetheless visible in that line. Table 2.5 states them in the form they take as design requirements, together with what this thesis does about each.

Gap 1 is not a hypothetical concern. An earlier version of the present work reported a large advantage for the constrained agent that was later traced to exactly this class of implementation artifact and withdrawn in full, as Chapter 4 records. Written as an identity, the difference prior work reports splits into two terms of different kinds:

$$\underbrace{Q(\text{cPPO}) - Q(\text{PPO})}_{\text{what is reported}} = \underbrace{[Q(\text{ctrl}) - Q(\text{PPO})]}_{\text{implementation term}} + \underbrace{[Q(\text{cPPO}) - Q(\text{ctrl})]}_{\text{constraint term}}, \quad (2.12)$$

where *ctrl* carries the cost critic with its multiplier pinned to zero. The design that resolves all three gaps is shown in Figure 2.6. One detail of it is worth stating in advance, because

Table 2.5. The three gaps and the design response to each.

	What the literature does	Why it is a problem	What this thesis does
Gap 1. No control arm	Trains cPPO against PPO and reports the difference [?]	cPPO also adds a cost-value network sharing an optimiser, buffer and RNG stream with the policy, so the difference sums an implementation effect and a constraint effect	Trains a control arm carrying the cost critic with λ pinned to zero, and measures the implementation term directly (Section 4.2)
Gap 2. No dispersion	Reports means; five of eight studies do not state a seed count	A practitioner deploys one policy, not a mean over runs they will never perform [?]	Trains five seeds per arm (1, 3, 4, 52, 54) and reports the spread of every quantity (Section 4.6)
Gap 3. One budget only	Fixes $d = 25$ and never varies it [?, ?]	A single budget cannot show whether the effect is a property of the constraint or an artifact of one threshold	Trains a second constrained arm at $d = 15$, so the effect can be read against how hard the constraint is applied (Section 3.10.1)

it is what makes Equation (2.12) usable rather than merely true. A plain PPO arm was trained alongside the control arm on the same seeds, and its stored network weights proved byte-identical to the control arm's. The implementation term is therefore measured at exactly zero rather than estimated, the control arm stands in for plain PPO throughout, and the arm labelled “PPO (baseline)” in Chapter 4 is that control arm. Section 4.2 gives the verification.

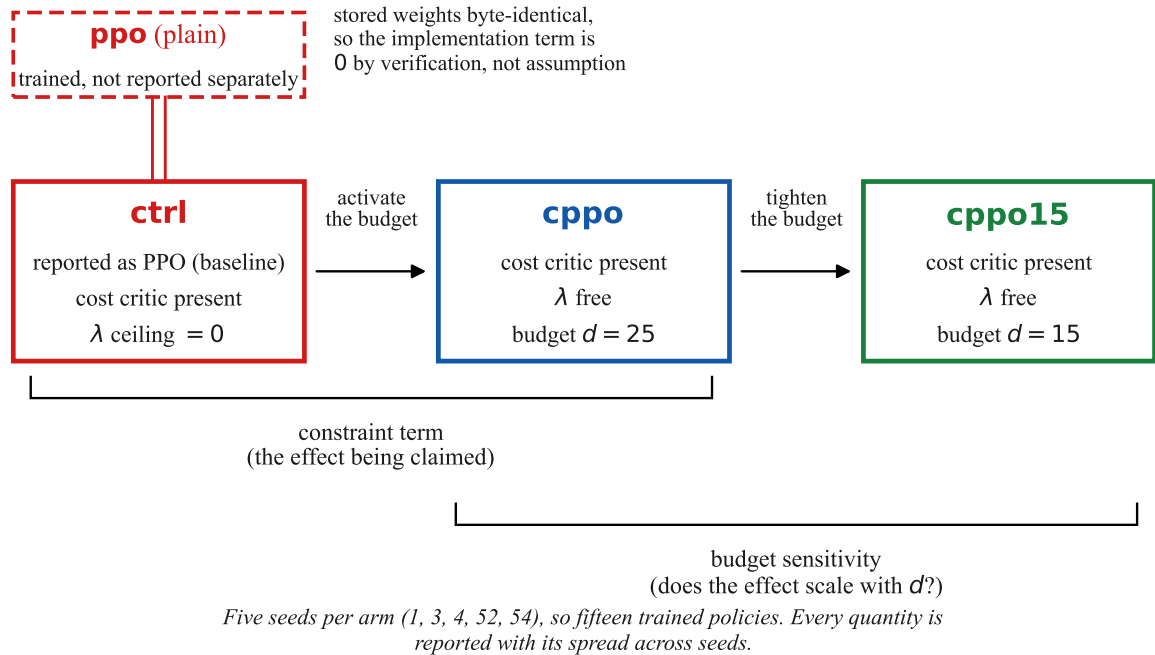


Figure 2.6. The three trained arms, and the two comparisons they support.

Table 2.6 sets that design against the two studies it most closely resembles.

Table 2.6. Positioning against the two nearest published studies.

Property	Khan <i>et al.</i> [?]	Shen <i>et al.</i> [?]	This thesis
Manipulator platform	Franka Panda	4-DOF planar	UR5e
Safety expressed as	Constraint	Reward term	Constraint
Safety quantity	Collision cost	Manipulability w	Manipulability w , joint limit, collision
Constrained algorithm	PPO- Lagrangian	SAC (uncon- strained)	PPO- Lagrangian
Cost-critic control arm	×	×	✓
Seed count stated	×	✓ (5)	✓ (5)
Per-seed dispersion reported	×	×	✓
Budget varied	× ($d = 25$ only)	n/a	✓ ($d = 25$ and 15)
Frozen-policy evaluation	×	✓	✓

The work is also positioned locally. It continues a line of manipulator research in the same laboratory, of which the immediately preceding study [?] implemented classical image-based visual servoing with CSRT tracking on a five-degree-of-freedom manipulator. Where that work addressed the perception and servoing loop with classical methods, the present work addresses the control policy with learned ones, and adds the safety constraint that classical visual servoing does not itself provide.

2.12 Summary

The path from the first section of this chapter to the last is short enough to retrace in one pass, and doing so makes clear why the experiment of Chapter 4 takes the form it does.

It begins with a mismatch. Reinforcement learning gives a designer one channel through which to say what the agent should do, and safety requirements do not fit down it comfortably. A hazard entered into the reward becomes a penalty, a penalty needs a weight, and that weight is an exchange rate between injury and task progress that nobody can derive [?, ?]. Section 2.9 produced the concrete instance: in the closest published treatment of singularity avoidance by learning, that exchange rate is $\lambda_3 = 0.05$, and the paper offers no account of where it came from [?]. That is not a lapse on the authors' part, since there is nothing available to derive it from.

The constrained Markov decision process resolves the mismatch by moving the requirement out of the objective and into a separate condition with a budget [?], an approach that Gu *et al.* [?] report has become the prevailing formalism. Its Lagrangian relaxation does eventually optimise a weighted sum, but the weight is now a dual variable driven by the measured

violation. Among the deep algorithms implementing this, PPO-Lagrangian was chosen over CPO on published evidence: on the Safety Gym slate CPO removed about forty percent of the unconstrained agent’s violations where the Lagrangian methods removed about ninety-seven [%]. That choice carries a known cost, which Stooke *et al.* [%] diagnose exactly. Plain dual ascent is integral control, integral control overshoots, and the multiplier and the cost consequently chase one another around the budget. Any λ reported in this thesis is a controller mid-flight, and the separate cost critic the same authors recommend is what later forces the control arm into the experimental design.

Turning to robotics narrowed the field quickly. Learned grasping on manipulators is well established and the numbers are respectable: 100 % grasp success with zero-shot hardware transfer on a Franka Panda [%], 87 % for SAC against 82 % for PPO on a UR5 with a Robotiq gripper [%], and 93 % dynamic obstacle avoidance on a UR5e in a human-robot cell [%]. None of them is a bound, however. They are success rates obtained under a shaped reward, and a success rate says nothing about the configurations the arm passed through on the way. Brunke *et al.* [%] supply the vocabulary joining the safety and manipulation literatures, along with the caveat this thesis must respect: a constraint in expectation bounds average exposure, not the severity of any one excursion. The operative quantity itself came from an older source. Yoshikawa’s measure $w = \sqrt{\det(JJ^T)}$ is the product of the Jacobian’s singular values and the volume of the velocity ellipsoid [%], which is why it collapses when the arm loses a single direction of motion instead of degrading gradually, and why a modest Cartesian command near that condition becomes a large joint rate on a real UR5e.

The last piece was methodological, and it changed the question instead of the method. Henderson *et al.* [%] showed that ten runs of one algorithm on one task, differing only in seed, split into two groups of five that are statistically distinct at $p = 0.0016$. Reading Table 2.4 in that light turned an incidental observation into a finding: of the eight experimental studies reviewed, five do not state a seed count and none reports per-seed values, so the published safety numbers in this area are means of unknown composition. Since a practitioner deploys one policy and not a mean, the useful property of a safety mechanism may be how tightly it constrains the spread of outcomes rather than where it puts their centre.

Those threads converge on the three gaps of Section 2.11, all visible in the study nearest this one [%]: no control arm separating the effect of adding a cost critic from the effect of activating the constraint, no dispersion behind the reported mean, and no result at any budget other than the inherited $d = 25$. None is a fault of that paper, since all three are field-wide conventions, and all three are answerable with experimental design and no new algorithm.

That is the scope this thesis takes on, and its boundaries should be stated as plainly as its content. It proposes no new optimiser: the constrained agent is PPO-Lagrangian as published [%], deliberately unmodified, because a modified algorithm would reintroduce the confound

the control arm exists to remove. It works in simulation, so its claims concern a simulated UR5e and are transferable only in the sense that [?] demonstrates such transfer is possible. It abstracts the grasp as a weld, so its success rates are not comparable with the full-grasp figures of [?]. And although manipulability is the constraint the argument is built on, Section 3.9 shows joint-limit proximity active alongside it, with which of the two consuming the budget varying from seed to seed, so the safety result is not a manipulability result alone.

Within those boundaries it contributes fifteen trained policies across three arms and five seeds: a control arm whose weights are verified identical to plain PPO, so the implementation term is measured at zero rather than assumed away; a constrained arm at the inherited budget; and a second constrained arm at a tighter one, so the effect can be read against how hard the constraint is applied. Every quantity is reported with its spread across seeds. Chapter 3 sets out the method by which that is done.